LatticeFlow: Cost-Refined Flow Matching over Motion Primitives for Depth-Based Quadrotor Navigation

Anonymous Authors

Abstract— Reactive quadrotor navigation must preserve multiple avoidance hypotheses while producing control-ready trajectories at onboard rates. We present LatticeFlow, a single-frame depth policy that transports a deterministic set of 15 physical motion-primitive anchors to cost-refined terminal position, velocity, and acceleration states. The network is trained from random initialization without policy distillation or a pretrained perception backbone. During training only, map-based distance fields and dense depth evaluate anchor, detached-student, and locally perturbed endpoint candidates; accepted gradient steps monotonically reduce their trajectory cost and define flow-matching targets. At deployment, the policy uses only projected depth, goal direction, velocity, and acceleration. On 20,000 queries from two held-out maps, LatticeFlow lowers selected trajectory cost by 8.31% and a 0.6-m collision proxy from 28.48% to 18.49% relative to a retrained direct lattice regressor. It reaches all five goals without collision in matched ROS simulation, while an optional one-step selector reduces lattice switching from 0.145 to 0.024. A sensor-matched TensorRT instance completes two collision-free LiDAR-only flights, reaches both commanded goal regions, records a maximum speed of 2.68 m/s, and averages 20.8-ms inference and 43.3-ms total planning time on an NVIDIA Orin NX. These real-flight results establish feasibility but remain descriptive because only two trials were conducted.

I. Introduction

Fast quadrotor flight through clutter requires local decisions from partial observations under tight compute and control constraints. Classical systems separate mapping, search, trajectory optimization, and control. This decomposition provides explicit structure, but repeated map updates and optimization can limit reaction speed. Learned planners instead map onboard observations directly to collision-aware motion and have enabled aggressive autonomous flight [1]. A practical reactive policy must still represent left, right, over, and under alternatives and avoid changing modes erratically between replanning instants.

One-stage lattice planners preserve such alternatives by predicting one terminal state and score per camera-aligned motion primitive [2]. Their output is efficient and directly convertible to polynomial trajectories, but one-shot regression does not model how a geometric prior is improved toward a lower-cost endpoint. Generative policies provide an alternative: diffusion policies iteratively denoise sampled actions [3], while flow matching learns a continuous vector field between source and target distributions [4]. Informative sources can reduce the burden of transport [5], yet action histories are unavailable in independently sampled navigation datasets, and resampling random sources can destabilize reactive control.

LatticeFlow uses motion primitives themselves as deterministic sources. Each cell starts from a different normalized physical terminal position with zero terminal velocity and acceleration. A depth- and state-conditioned vector field then transports all cells to cost-refined terminal states. The targets do not come from a teacher: anchors, detached current predictions, and local perturbations are ranked by privileged trajectory costs and improved by accepted gradient steps. The resulting policy is multimodal across lattice cells but deterministic within each cell, so identical inputs yield identical candidate trajectories.

The deployment boundary is equally important. The neural policy receives one depth image plus goal direction, velocity, and acceleration; maps, point clouds, and distance fields are used only to construct training costs. For LiDAR deployment, registered points are projected into the trained camera model and no-return statistics are matched using real return masks. This avoids treating a deterministic sensor-model mismatch as a generic sim-to-real failure.

Our contributions are:

- an explicit physical-anchor flow formulation whose source states are distinct terminal position–velocity–acceleration tuples and whose output remains directly decodable into fifth-order trajectories;
- a teacher-free target-search algorithm that combines detached student exploration with monotonic privileged-cost refinement from fully random network initialization;
- local consistency regularization and an optional one-step continuity selector, evaluated separately so deterministic flow is not conflated with temporal mode stability; and
- leakage-controlled evaluation on 20,000 held-out queries, matched ROS closed-loop trials, sensor-domain ablations, and two successful LiDAR-only real flights with onboard timing.

II. Related Work

A. Vision-Based Quadrotor Planning

Classical local planners combine mapping, kinodynamic search, and polynomial or spline optimization. Topology-guided and fast planners obtain trajectories from distinct path classes or front-end seeds [6], [7], [8], but maintain an explicit world representation and

repeatedly optimize online. Learning-based alternatives predict collision likelihoods, commands, or local trajectories from onboard sensing. Representative systems include simulation-to-real single-image avoidance [9], reaction-based local planning [10], learned perception combined with minimum-time planning [11], high-speed flight in natural environments [1], and perception-aware trajectories for dynamic scenes [12]. One-stage spherical-lattice prediction further unifies perception, primitive selection, and terminal-state regression [2]; our focus is instead how to transport structured primitive anchors toward self-improved endpoints.

B. Motion Primitives and Structured Multimodality

Motion primitives discretize local planning into dynamically meaningful hypotheses and support efficient feasibility evaluation [13]. A fixed bank can be restrictive, whereas residual terminal-state prediction adapts each primitive to the observation. We reinterpret this bank as a conditional source set for a neural ODE. Multimodality is carried by different cells rather than by repeated random sampling within a cell, which is well suited to deterministic high-rate replanning.

C. Diffusion, Flow, and Policy Improvement

Diffusion Policy represents multimodal robot behavior through iterative denoising [3]. Flow matching directly regresses a time-dependent velocity field [4], and rectified flow favors efficient near-straight transport [14]. A2A replaces Gaussian noise with historical actions to predict future action latents [5]. We adopt the informative-source principle but use geometry when histories are absent. We also replace demonstration targets with detached policy proposals refined by training-only trajectory costs. Because continuous within-cell transport does not prevent discrete score changes between cells, continuity regularization and post-network selection are treated as distinct mechanisms.

III. Method

A. Problem and Physical Lattice Source

At planning time t, the policy receives a normalized depth image $D_t \in \mathbb{R}^{1\times 96\times 160}$ and body-frame state

$$s_t = [v_t, a_t, g_t] \in \mathbb{R}^9, \tag{1}$$

where v_t, a_t, and g_t are velocity, acceleration, and goal direction. No map, point cloud, distance field, previous image, or previous action enters the neural policy at deployment.

Inspired by the spherical motion-primitive construction of YOPO [2], we use a 3×5 camera-aligned lattice with $L = 15$ cells. A normalized primitive residual

$$r^\ell = [\Delta\theta, \Delta\phi, \Delta r, \tilde{v}_T^\top, \tilde{a}_T^\top]^\top \in [-1, 1]^9 \tag{2}$$

is mapped by a differentiable cell transform ψ_ℓ to body-frame terminal state $[p_T, v_T, a_T]$. This terminal state and the current state uniquely define a fifth-order polynomial over a fixed horizon.

Our flow operates in normalized physical coordinates

$$z^\ell = S\psi_\ell(r^\ell) = [p_T^\top/s_p, v_T^\top/s_v, a_T^\top/s_a]^\top, \tag{3}$$

with $s_p = 10$ m and $s_v = s_a = 6\sqrt{3}$. Let p^ℓ_{anc} be the position decoded at $r^\ell = 0$. The source is

$$z_0^\ell = [(p^\ell_{\text{anc}})^\top/s_p, 0, 0]^\top. \tag{4}$$

Consequently, the 15 ODE states differ physically before any learned update. After integration, z_K^ℓ is mapped back through $r_K^\ell = \psi_\ell^{-1}(S^{-1}z_K^\ell)$ for trajectory reconstruction and control.

B. Teacher-Free Cost-Refined Targets

All neural weights, including the depth backbone, are randomly initialized. The last flow layer is initialized to zero, so the initial policy preserves Eq. (4). For each cell, the current policy supplies a detached proposal r_S^ℓ. We construct

$$\mathcal{C}^\ell = \{0, \text{sg}(r_S^\ell), \epsilon_{A,1}, \ldots, \epsilon_{A,M_A}, \text{sg}(r_S^\ell) + \epsilon_{S,1}, \ldots\}, \tag{5}$$

where 0 decodes to the physical anchor, sg denotes stop-gradient, and the clipped Gaussian perturbations are used only for training-time exploration. Candidates are ranked independently per cell by

$$J = J_{\text{smooth}} + J_{\text{acc}} + J_{\text{obs}} + J_{\text{goal}} + \lambda_d J_{\text{depth}}. \tag{6}$$

J_{obs} queries a map-derived Euclidean signed distance field (ESDF). For body-frame polynomial samples transformed into the depth-camera frame, the image term uses

$$u_n = c_x - f_x y_n/x_n, \qquad v_n = c_y - f_y z_n/x_n, \tag{7}$$

$$c_{d,n} = d_{\max} D_t(u_n, v_n) - x_n, \tag{8}$$

$$\rho(c) = \beta \log\left(1 + e^{(d_s - c)/\beta}\right), \tag{9}$$

$$J_{\text{depth}} = N_v^{-1} \sum_{n\in\mathcal{V}} \rho(c_{d,n}). \tag{10}$$

Starting from the minimum-cost candidate $\bar{r}^\ell$, we propose normalized cost-gradient steps

$$r^{\ell,+} = \text{clip}\left(\bar{r}^\ell - \eta \frac{\nabla_{\bar{r}^\ell} \log(1+J)}{\text{RMS}_c(\nabla_{\bar{r}^\ell} \log(1+J)) + \varepsilon}\right). \tag{11}$$

A proposal replaces the current target only when its uncompressed trajectory cost decreases. The final target is detached and converted to $z_1^\ell = S\psi_\ell(r_1^\ell)$. Maps and ESDFs therefore supervise policy improvement but are absent at inference.

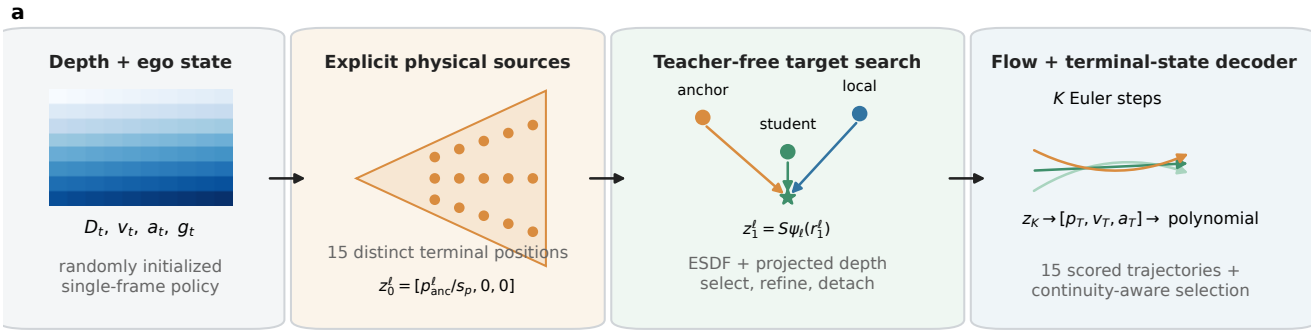

Fig. 1. LatticeFlow training and inference. Fifteen physical terminal-state anchors form deterministic source hypotheses. During training only, anchors, detached student predictions, and local perturbations are evaluated and monotonically refined by privileged trajectory costs. The conditional flow transports each anchor to a refined terminal state, which is decoded into a fifth-order polynomial and scored for selection.

C. Conditional Flow and Training Objective

A convolutional depth backbone produces a 3×5 feature map. Velocity, acceleration, and goal are normalized and expressed in each primitive frame. A shared pointwise MLP conditions on the cell feature, transformed state, current physical flow state, sinusoidal time embedding, and learned lattice embedding:

$$v_\theta^\ell = v_\theta(z_\tau^\ell, \tau, D_t, s_t, e_\ell). \tag{12}$$

For $\tau \sim \mathcal{U}(0,1)$, the straight conditional path and target velocity are

$$z_\tau^\ell = (1-\tau) z_0^\ell + \tau z_1^\ell, \tag{13}$$
$$u_\tau^\ell = z_1^\ell - z_0^\ell, \tag{14}$$

and

$$\mathcal{L}_{\mathrm{FM}} = \mathbb{E}_{\tau,\ell} \| v_\theta^\ell - u_\tau^\ell \|_2^2. \tag{15}$$

Euler integration uses K neural-function evaluations (NFE). A second head predicts the log-compressed trajectory cost of each integrated endpoint. The complete loss is

$$\mathcal{L} = \lambda_f \mathcal{L}_{\mathrm{FM}} + \lambda_e \mathcal{L}_{\mathrm{end}} + \lambda_J \log(1+J) + \lambda_s \mathcal{L}_{\mathrm{score}} + \lambda_c \mathcal{L}_{\mathrm{cons}} + \lambda_l \mathcal{L}_{\mathrm{lat}}. \tag{16}$$

$\mathcal{L}_{\mathrm{cons}}$ applies Huber consistency to endpoints and centered scores under small depth/state perturbations. $\mathcal{L}_{\mathrm{lat}}$ penalizes horizontal and vertical second differences across neighboring cells. These terms encourage local robustness; they are not temporal supervision because training samples are independent.

The source-noise variance is zero. Thus a fixed input deterministically produces one trajectory per cell, and the 15 cells carry the avoidance alternatives. This structured flow does not itself guarantee that the minimum-score cell remains unchanged over time.

D. Optional Continuity-Aware Selection

For deployments that favor command continuity, a one-step selector stores the previously selected endpoint p_{t-1}^* and adjusts each current score:

$$\hat{q}_t^\ell = q_t^\ell + \lambda_p \| p_t^\ell - p_{t-1}^* \|_2. \tag{17}$$

The previous cell is retained while its raw score is within margin δ_q of the current raw minimum; otherwise the minimum adjusted score is selected. This state is outside the neural policy and is ablated separately. The real flights reported below use raw selection with this selector disabled.

IV. Experimental Setup

A. Controlled Training and Offline Evaluation

The standard-view dataset contains ten independently generated simulated forest maps, each with 10,000 depth images, camera poses, and a point cloud used only for privileged cost computation. Maps 0–6 are used for training, map 7 for model selection, and maps 8–9 only for final testing. Validation and test ego states are deterministically generated per image.

The main model is trained for 20 epochs from random weights with batch size 16, AdamW, learning rate 1.5×10^{-4}, two anchor perturbations, one student perturbation, two accepted refinement steps, and six NFE. Every batch logs total and component losses, gradient norm, target-source fractions, target improvement, and acceptance rate to TensorBoard. The depth backbone is not pretrained. All flow variants contain 11.469 million trainable parameters.

We compare against a direct one-stage lattice regressor retrained on the same map split and budget. For diagnosis, residual-coordinate and physical-coordinate flow variants additionally use a frozen direct policy for initial backbone weights and target proposals. These teacher-guided models quantify the value of teacher information but are not part of the proposed training pipeline. All

learned runs use seed 0; training-seed variance is not estimated.

All 20,000 fixed queries from held-out maps 8–9 are evaluated with identical perturbations. Metrics include selected and oracle cost, projected clearance, a 0.6-m collision proxy, score regret, cell switching under small input perturbations, endpoint shift, and full-network latency. Query-level bootstrap intervals are descriptive because samples are nested within only two maps; per-map means are also reported.

B. ROS Closed Loop

Closed-loop trials use the CUDA depth simulator, fifth-order reconstruction, an SO(3) controller, and geometric collision checking against the captured forest point cloud. Simulator seed 3, start $[2, -2, 2]$ m, and five goals are fixed across methods. Success requires entering a 1-m goal ball within 60 s; collision is clearance below 0.3 m. Each policy–goal pair restarts from the same state. With five rollouts per configuration on one map, the outcomes are descriptive.

C. LiDAR Perception-Domain Matching

The real platform has a Livox MID-360 rather than a depth camera. We therefore render a second 100,000-frame dataset with the sensor's elevated field of view: 90° horizontal and approximately −7° to +52° vertical, represented by a +22.5° optical-axis elevation. The same map split and 20-epoch training protocol are retained. Each dense simulator image is converted to the deployed input domain using one of 120 recorded raw-return masks, one 3×3 local hole-fill iteration requiring five valid neighbors, a world-frame $z = 2$ m virtual-ceiling projection at stride two, 20-m no-return depth, range noise, and quantization. The ceiling is an image-space sensing prior, not a trajectory-height constraint. Dense depth, point clouds, and ESDFs remain training-only privileged signals.

The deployed model is a separately trained, sensor-matched instance of the same teacher-free architecture. It is exported to FP16 TensorRT with six NFE and runs on an NVIDIA Orin NX. Registered LiDAR points and LIO odometry are projected online to a 96×160 depth image; the output terminal state is reconstructed and published to the existing position-command controller.

D. Real-Flight Protocol

We analyze two independently recorded obstacle-avoidance flights with nominal forward goals of 4 and 8 m. Both use raw minimum-score selection; the continuity selector is disabled. Success is final odometry within 1 m of the commanded goal with no observed physical contact. Bag analysis uses all command-mode odometry and controller samples. For visualization only, accumulated LiDAR returns are voxelized at 8 cm and restricted to $z \in [0.3, 2.5]$ m so ground and overhead returns do not obscure obstacles at flight height. Because $n = 2$ and no real-flight baseline was run, these trials test feasibility rather than comparative superiority.

TABLE I

Held-out maps 8–9. T: teacher-guided diagnostic. Lower is better except clearance.

Metric	Direct	Residual-T	Physical-T	Ours
Selected cost	3.1415	2.8622	2.8611	2.8805
Oracle cost	2.7946	2.7565	2.7629	2.7736
Clearance (m)	1.9894	3.1433	3.0385	3.0421
Collision proxy	28.48%	18.15%	18.90%	18.49%
Perturbation switch	4.51%	3.38%	3.22%	3.38%
Endpoint shift (m)	0.1161	0.1011	0.0941	0.1067
Selection regret	0.3469	0.1057	0.0983	0.1069

V. Results

A. Held-Out Trajectory Quality

LatticeFlow reduces selected cost from 3.1415 for direct lattice regression to 2.8805, an 8.31% decrease (Table I, Fig. 2). The paired query-level difference is −0.2610 with a descriptive 95% interval of $[-0.2728, -0.2496]$. The decrease appears on both maps: 2.8566 versus 3.1046 on map 8 and 2.9044 versus 3.1783 on map 9.

Mean projected clearance increases from 1.989 to 3.042 m, while the collision proxy decreases from 28.48% to 18.49%. Perturbation switching decreases from 4.51% to 3.38%, and endpoint shift decreases from 0.1161 to 0.1067 m. These results show that privileged-cost self-improvement can train the complete policy without a pretrained backbone or teacher proposals.

Teacher information yields a small cost advantage but is not required for the principal gain. Relative to physical-coordinate teacher guidance, the teacher-free selected cost is 0.0193 higher (descriptive 95% interval $[0.0156, 0.0231]$), whereas its collision proxy is 0.41 percentage points lower. Clearance is nearly unchanged, but endpoint shift and regret are 0.0126 m and 0.0086 higher. Late in training, detached student proposals dominate the selected target seeds and accepted gradient steps continue to lower cost, consistent with self-improvement rather than fixed imitation. Only one training seed was evaluated.

Four to six NFE capture nearly all of the flow gain. Selected cost is 2.9519, 2.8916, 2.8823, 2.8805, and 2.8805 for 1, 2, 4, 6, and 8 NFE. Corresponding RTX 5070 Ti full-network latency is 1.540, 1.645, 2.014, 2.616, and 2.646 ms per batch of 16. Thus four NFE increases cost by only 0.065% relative to six.

B. Closed-Loop Continuity

Every reported ROS configuration reaches all five goals without geometric collision (Table II). Raw LatticeFlow is fastest at 8.81 s but exhibits a 0.145 cell-switch rate, confirming that deterministic within-cell flow does not ensure temporal mode consistency. The optional selector reduces switching to 0.024, endpoint jump from 0.271 to 0.164 m, and jerk from 6.45 to 5.56 m/s^3, with

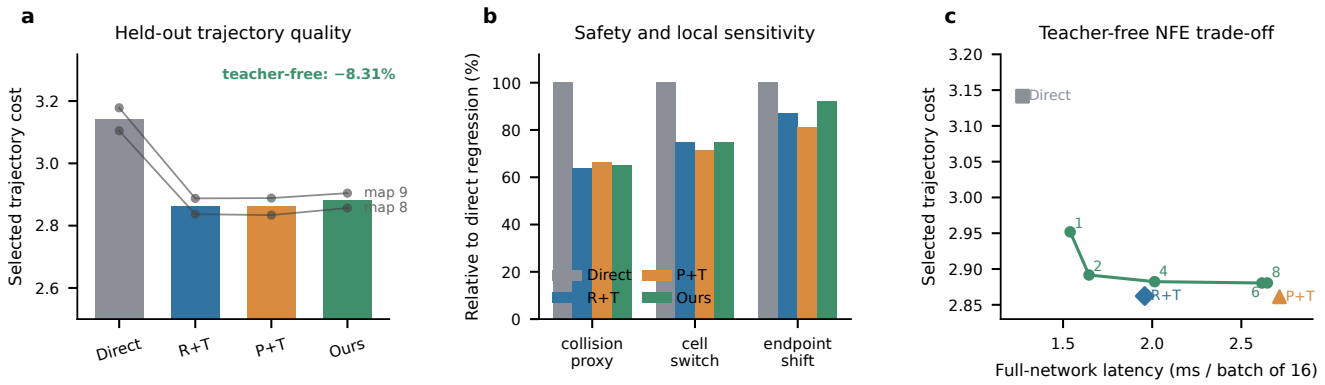

Fig. 2. Held-out evaluation. (a) Selected cost overall and on each map. R+T and P+T are teacher-guided residual- and physical-coordinate diagnostics; Ours is teacher-free physical flow. (b) Safety and local-sensitivity metrics normalized to direct lattice regression. (c) NFE–latency trade-off for the teacher-free model with six-NFE diagnostic references. All means use 20,000 fixed queries.

TABLE II

Matched ROS trials on one forest, five goals per configuration.

Method	Succ.	Time (s)	Switch	Jump (m)	Jerk (m/s^3)	Net (ms)
Direct raw	5/5	9.48	0.144	0.239	6.10	1.17
Residual-T + sel.	5/5	9.73	0.026	0.152	4.72	2.18
Physical-T + sel.	5/5	9.67	0.028	0.164	5.28	3.25
Ours raw	5/5	8.81	0.145	0.271	6.45	3.26
Ours + sel.	5/5	9.62	0.024	0.164	5.56	3.24

mean arrival time increasing to 9.62 s. A teacher-guided residual diagnostic remains smoother in endpoint jump and jerk, so the proposed teacher-free model does not dominate every continuity metric.

C. Perception Alignment and Real Flight

Sensor-domain matching is necessary for this deployment. On the same 20,000 sparse MID-360 held-out inputs, the standard-view model obtains 5.1706 selected cost and 57.25% collision proxy; native-FOV training with recorded return masks improves them to 2.9169 and 13.43%. On 120 recorded real depth frames with a level 4-m goal, the mean complete-trajectory vertical envelope decreases from 0.583 to 0.112 m, and frames within ±0.3 m increase from 0/120 to 120/120.

Removing the virtual ceiling during training does not materially change matched-domain held-out cost (2.9200 without ceiling versus 2.9170 with ceiling), but adding the ceiling only at inference increases cost to 3.2923 and produces upward excursions above 0.3 m in 96.7% of the recorded frames. The relevant conclusion is train–deployment perception consistency, not that a virtual ceiling is universally beneficial.

Both real flights complete collision-free obstacle avoidance and enter the 1-m goal region (Fig. 3, Table III). The 4-m trial follows a 5.58-m path in 15.17 s and ends 0.88 m from its goal. The 8-m trial follows a 9.50-m path in 8.08 s, ends 0.55 m from its goal, and reaches 2.68 m/s. Altitude remains within 0.798–1.464 m across both trials. Mean inference is 20.8 ms and mean total planning time is 43.3 ms on the Orin NX. These measurements demonstrate onboard feasibility, but two successful flights without a real baseline cannot establish a success rate or comparative safety advantage.

VI. Conclusion

LatticeFlow turns a deterministic bank of physical motion-primitive anchors into a teacher-free conditional flow policy for depth-based quadrotor navigation. Privileged ESDF and dense-depth costs select and monotonically refine detached student targets during training, while deployment uses only projected depth and ego state. The model improves held-out cost and collision proxies over direct lattice regression, succeeds in matched ROS trials, and a sensor-aligned instance completes two LiDAR-only real flights with onboard TensorRT inference. The evidence supports structured physical sources and privileged-cost self-improvement as a practical planning recipe, but it does not yet isolate flow integration from all component-matched regression alternatives. Important next steps are multiple training seeds, more simulator maps and failure-inducing scenarios, a same-backbone direct target-regression control, dynamic obstacles, and substantially more real flights with matched baselines.

Generative AI Disclosure

The authors used OpenAI Codex to assist with code implementation, experiment orchestration, and language drafting. All technical decisions, experimental outputs, citations, and the final manuscript were checked by the human authors.

References

[1] A. Loquercio, E. Kaufmann, R. Ranftl, M. Müller, V. Koltun, and D. Scaramuzza, "Learning high-speed flight in the wild," Science Robotics, vol. 6, no. 59, p. eabg5810, 2021.

[2] J. Lu, X. Zhang, H. Shen, L. Xu, and B. Tian, "You only plan once: A learning-based one-stage planner with guidance learning," IEEE Robotics and Automation Letters, vol. 9, no. 7, pp. 6083–6090, 2024.

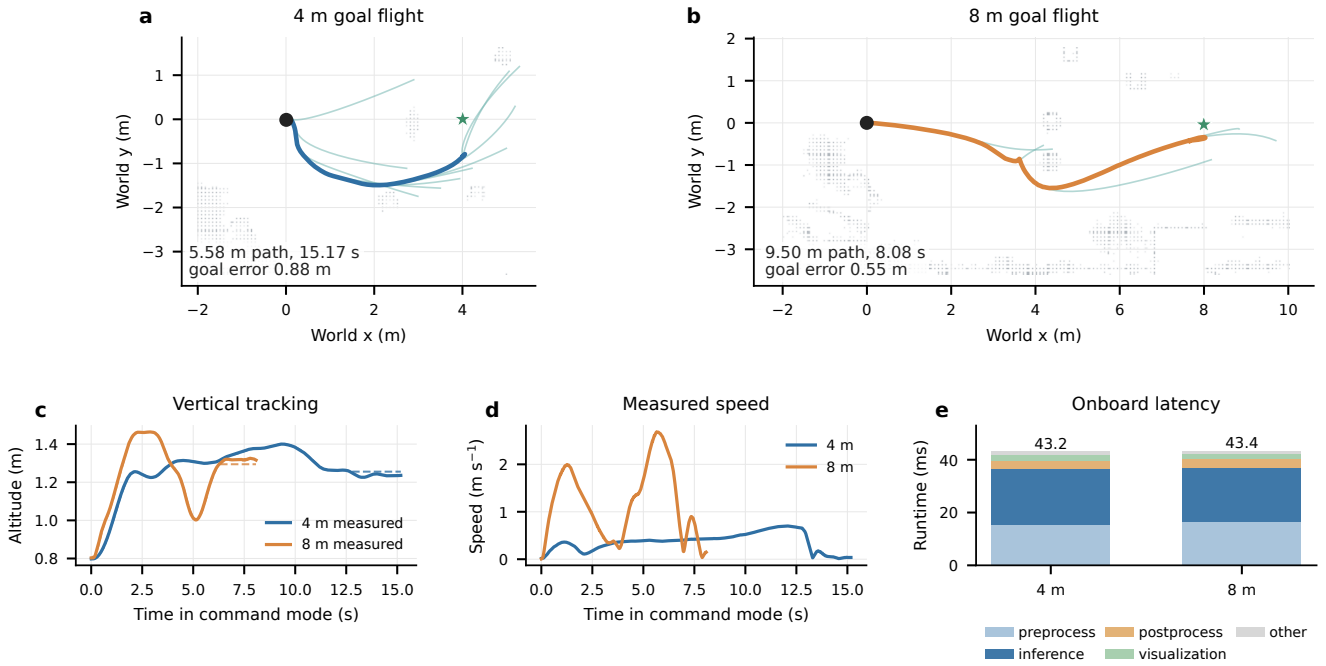

Fig. 3. LiDAR-only real-flight evidence. (a,b) Complete measured paths, sampled planned trajectories, goals, and accumulated voxelized LiDAR returns for 4-m and 8-m commands. (c) Measured and commanded altitude. (d) Measured speed. (e) Mean onboard timing decomposition. Both flights used TensorRT, six NFE, and raw lattice selection.

TABLE III

Real-flight bag metrics. Both trials succeeded without observed collision; selector disabled.

Trial	Goal (m)	Duration (s)	Path (m)	$v_{\max}$ (m/s)	Goal error (m)	Altitude (m)	Inference / total (ms)	Outcome
Flight 1	4	15.17	5.58	0.70	0.88	0.798–1.400	21.12 / 43.18	success
Flight 2	8	8.08	9.50	2.68	0.55	0.803–1.464	20.50 / 43.39	success

[3] C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel, R. Tedrake, and S. Song, "Diffusion policy: Visuomotor policy learning via action diffusion," in Robotics: Science and Systems, 2023.

[4] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le, "Flow matching for generative modeling," in International Conference on Learning Representations, 2023.

[5] J. Jia, G. Li, X. Chen, T. An, Y. Hu, J. Li, X. Guo, and J. Yang, "Action-to-action flow matching," arXiv preprint arXiv:2602.07322, 2026.

[6] B. Zhou, F. Gao, J. Pan, and S. Shen, "Robust real-time UAV replanning using guided gradient-based optimization and topological paths," in IEEE International Conference on Robotics and Automation, 2019, pp. 1208–1214.

[7] H. Ye, X. Zhou, Z. Wang, C. Xu, J. Chu, and F. Gao, "TGK-planner: An efficient topology guided kinodynamic planner for autonomous quadrotors," IEEE Robotics and Automation Letters, vol. 6, no. 2, pp. 494–501, 2021.

[8] B. Zhou, F. Gao, L. Wang, C. Liu, and S. Shen, "Robust and efficient quadrotor trajectory generation for fast autonomous flight," IEEE Robotics and Automation Letters, vol. 4, no. 4, pp. 3529–3536, 2019.

[9] F. Sadeghi and S. Levine, "CAD2RL: Real single-image flight without a single real image," in Robotics: Science and Systems, 2017.

[10] J. Lu, B. Tian, H. Shen, X. Zhang, and Y. Hui, "LPNet: A reaction-based local planner for autonomous collision avoidance using imitation learning," IEEE Robotics and Automation Letters, vol. 8, no. 11, pp. 7058–7065, 2023.

[11] R. Penicka, Y. Song, E. Kaufmann, and D. Scaramuzza, "Learning minimum-time flight in cluttered environments," IEEE Robotics and Automation Letters, vol. 7, no. 3, pp. 7209–7216, 2022.

[12] J. Tordesillas and J. P. How, "Deep-PANTHER: Learning-based perception-aware trajectory planner in dynamic environments," IEEE Robotics and Automation Letters, vol. 8, no. 3, pp. 1399–1406, 2023.

[13] H. Nguyen, S. H. Fyhn, P. D. Petris, and K. Alexis, "Motion primitives-based navigation planning using deep collision prediction," in IEEE International Conference on Robotics and Automation, 2022, pp. 9660–9667.

[14] X. Liu, C. Gong, and Q. Liu, "Flow straight and fast: Learning to generate and transfer data with rectified flow," in International Conference on Learning Representations, 2023.